

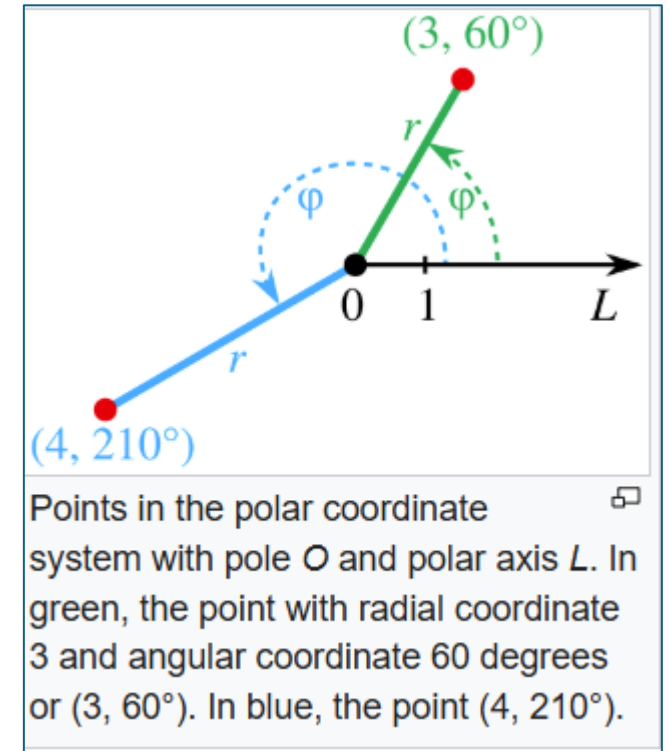
Polars

Python Library Overview

By : Genti Dallvo

What is Polars Library

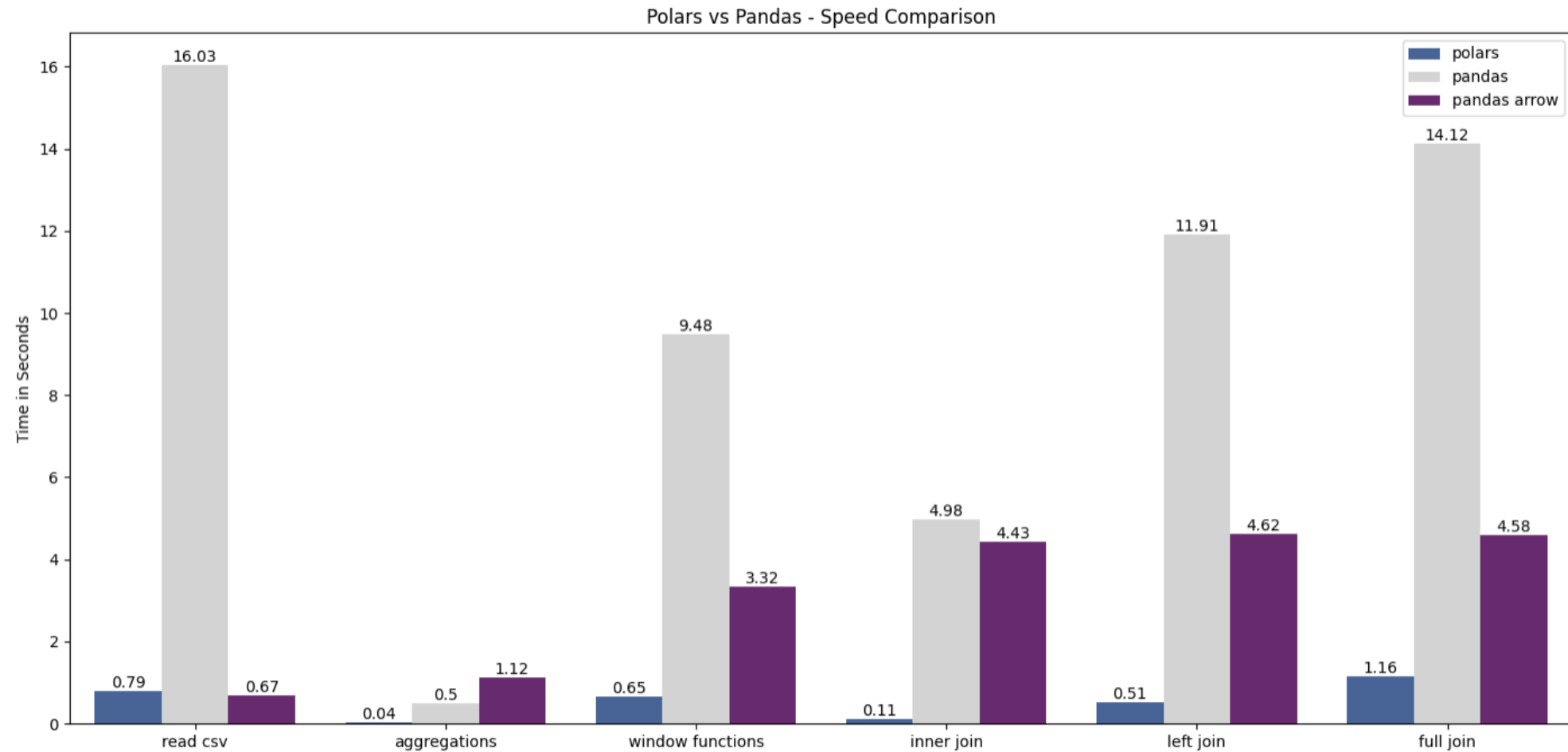
- The name "Polars" for the Python data frame library originates from its core design principle of **leveraging CPU parallelism**. Just as polar coordinates define a plane using angles and a radius extending outwards (radially) from a central point, Polars is designed to process data by radiating computations outwards across multiple CPU cores simultaneously.
- The Polars library in Python is a high-performance data manipulation library designed primarily for **use in data science, data analysis, and machine learning contexts**. It provides functionalities similar to pandas but is optimized to handle large datasets more efficiently.
- **It is written in Rust** – a system programming language designed for safety, speed, and concurrency particularly when working with large datasets. Polars utilizes a multi-threaded query engine and columnar storage, enabling parallel processing and optimized memory usage. It supports **lazy evaluation**, optimizing queries for better performance and reduced memory consumption.



Key Characteristics of Polars

- **Fast:** Written from scratch in Rust, designed close to the machine and without external dependencies.
- **I/O:** First class support for all common data storage layers: local, cloud storage & databases.
- **Intuitive API:** Write your queries the way they were intended. Polars, internally, will determine the most efficient way to execute using its query optimizer.
- **Out of Core:** The streaming API allows you to process your results without requiring all your data to be in memory at the same time.
- **Parallel:** Utilizes the power of your machine by dividing the workload among the available CPU cores without any additional configuration.
- **GPU Support:** Optionally run queries on NVIDIA GPUs for maximum performance for in-memory workloads.

Performance Comparison Results



Polars Use Cases

- **Financial Data Analysis** - Polars can be used to analyze large financial datasets, such as stock prices or transaction data, to identify trends, calculate moving averages, and perform other calculations.
- **Log File Analysis** - Polars can efficiently process and analyze large log files to extract relevant information, identify errors, and monitor system performance.
- **ETL Processes** - Polars can be used to extract, transform, and load data from various sources, making it suitable for building data pipelines
- **Data Cleaning and Preprocessing** - Polars provides tools for cleaning and preprocessing data, such as handling missing values, filtering rows, and transforming columns.
- **Large Dataset Handling** - Polars can handle datasets larger than available RAM due to its ability to work with data in chunks and its optimized query execution.
- **Geospatial Analysis** - Polars, along with plugins like polars-st, can be used for geospatial operations on dataframes, such as calculating distances or performing spatial joins.
- **String Parsing and Manipulation** - Plugins like polars-url and polars_iptools, can parse and extract information from URLs and IP addresses.
- **Fuzzy Matching and Text Similarity** - Plugins like polars-distance and polars-fuzzy-match, can be used for fuzzy matching and calculating text similarity between strings.

How is Polars Different than Pandas

- Pandas and Polars are both popular data manipulation libraries in Python, but they are designed with different focuses and underlying architectures. Here's a comparison of some key differences between pandas and Polars:

1. Performance and Scalability:

1. **Pandas:** It's widely used for its ease of use and flexibility but can **struggle with performance issues**, especially with very large datasets. Its performance typically degrades as the size of the data grows beyond the memory capacity of a single machine.
2. **Polars:** Designed to **handle large datasets more efficiently**, it uses a **multi-threaded approach** which gives it speed and memory efficiency advantages over pandas. Polars is often faster for large datasets and complex data manipulations.

2. API and Ease of Use:

1. **Pandas:** Has a well-established, intuitive API that is deeply integrated into the Python data science ecosystem. It offers a broad range of functionalities and is often the **first choice for data manipulation** tasks in Python.
2. **Polars:** While it provides an API similar to pandas to ease transition, it also introduces a unique expression **system for lazy computations**, which can be more verbose but allows for more optimized workflows.

How is Polars Different than Pandas

1. Data Handling and Storage:

1. **Pandas:** Uses an **in-memory row-based DataFrame which can lead to inefficiencies** when dealing with large datasets or when performing operations that are naturally columnar.
2. **Polars:** Utilizes a **columnar storage format**, which is more efficient for many types of operations and is better for optimizing memory usage during computations.

2. Concurrency and Parallelism:

1. **Pandas:** Operates in a **single-threaded** manner, which means it does not naturally take advantage of multi-core CPUs without external tools like Dask.
2. **Polars:** Built from the ground up to support **multi-threading**, allowing it to perform operations in parallel and thus significantly speeding up data processing tasks on multi-core machines.

3. Lazy Evaluation:

1. **Pandas:** Executes operations immediately when called, which can be **inefficient in cases where multiple operations are chained together**.
2. **Polars:** Supports lazy evaluation where operations are not executed immediately but are **optimized and executed only when needed**. This can lead to better performance especially in complex data pipelines.

4. Memory Management:

1. **Pandas:** Can **sometimes use memory inefficiently**, particularly with operations that involve copying data.
2. **Polars:** More **efficient with memory due to its columnar approach** and built-in optimizations that minimize unnecessary data duplication.

Function Comparison

Data Loading / I/O

Operation	Pandas	Polars
Read CSV	pd.read_csv()	pl.read_csv()
Read JSON	pd.read_json()	pl.read_json()
Write CSV	df.to_csv()	df.write_csv()
Write Parquet	df.to_parquet()	df.write_parquet()

Data Selection & Filtering

Operation	Pandas	Polars
Column select	df['col']	df.select('col') or df['col']
Row filter	df[df['col'] > 10]	df.filter(pl.col('col') > 10)
Multiple filters	df[(cond1) & (cond2)]	df.filter((cond1) & (cond2))

DataFrame Creation & Info

Operation	Pandas	Polars
Create DataFrame	pd.DataFrame(data)	pl.DataFrame(data)
Show schema / types	df.dtypes	df.schema
Head / Tail	df.head(), df.tail()	Same in Polars

Joining & Merging

Operation	Pandas	Polars
Merge	pd.merge(df1, df2, ...)	df1.join(df2, on=..., how='...')
Join Types	'inner', 'left', etc.	Same
As-of Join	pd.merge_asof()	df1.join_asof(df2, ...)

Summary

- In summary, while pandas is an excellent tool for many data manipulation tasks, particularly for small to moderately sized datasets, Polars offers significant advantages in terms of performance and efficiency for larger datasets or when parallel processing capabilities are needed. Depending on the specific requirements of a project, one might be more suitable than the other.

References

- <https://docs.pola.rs/#key-features>
- <https://realpython.com/polars-python/>
- <https://cliffy-gardens.medium.com/using-python-polars-my-late-2024-review-da806618a102>
- <https://stuffbyyuki.com/wp-content/uploads/2023/06/polars-vs-pandas-arrow-vs-pandas-1.png>
- https://en.wikipedia.org/wiki/Polar_coordinate_system#:~:text=The%20distance%20from%20the%20pole,to%20model%20using%20polar%20coordinates.

Apendix

- The goal of Polars is to provide a lightning fast DataFrame library that:
- Utilizes all available cores on your machine.
- Optimizes queries to reduce unneeded work/memory allocations.
- Handles datasets much larger than your available RAM.
- A consistent and predictable API.
- Adheres to a strict schema (data-types should be known before running the query).
- Polars is written in Rust which gives it C/C++ performance and allows it to fully control performance-critical parts in a query engine.